

```

import { getDatabase } from "@netlify/database";
import { verifyOwnerToken } from "../lib/auth.js";

const db = getDatabase();

function makeInvoiceNumber(n) {
  return `INV-${String(n).padStart(4, "0")}`;
}

export default async (req) => {
  const authed = await verifyOwnerToken(req);
  if (!authed) {
    return new Response(JSON.stringify({ error: "Not authenticated" }), { status:
  }

  const url = new URL(req.url);
  const resource = url.searchParams.get("resource"); // "clients" | "jobs" | "inv

  try {
    if (req.method === "GET") {
      const [clients, jobs, invoices] = await Promise.all([
        db.sql`SELECT * FROM clients ORDER BY name`,
        db.sql`SELECT * FROM jobs ORDER BY date, time`,
        db.sql`SELECT * FROM invoices ORDER BY number DESC`,
      ]);
      return Response.json({ clients, jobs, invoices });
    }

    if (req.method === "POST") {
      const body = await req.json();

      if (resource === "clients") {
        const code = Math.random().toString(36).slice(2, 8).toUpperCase();
        const [client] = await db.sql`
          INSERT INTO clients (name, address, phone, notes, access_code)
          VALUES (${body.name}, ${body.address || ""}, ${body.phone || ""}, ${bod
          RETURNING *
        `;
        return Response.json(client);
      }

      if (resource === "jobs") {
        const [job] = await db.sql`
          INSERT INTO jobs (client_id, date, time, type, price, status, notes)

```

```

VALUES (${body.clientId}, ${body.date}, ${body.time}, ${body.type}, ${b
RETURNING *
`;

// Auto-generate invoice if job is created as "done"
if (job.status === "done") {
  await createInvoiceForJob(job);
}
return Response.json(job);
}

return new Response("Unknown resource", { status: 400 });
}

if (req.method === "PATCH") {
  const body = await req.json();
  const id = url.searchParams.get("id");

  if (resource === "jobs") {
    const patch = {
      clientId: body.clientId ?? null,
      date: body.date ?? null,
      time: body.time ?? null,
      type: body.type ?? null,
      price: body.price ?? null,
      status: body.status ?? null,
      notes: body.notes ?? null,
    };
    const [job] = await db.sql`
      UPDATE jobs SET
        client_id = COALESCE(${patch.clientId}::uuid, client_id),
        date = COALESCE(${patch.date}::date, date),
        time = COALESCE(${patch.time}, time),
        type = COALESCE(${patch.type}, type),
        price = COALESCE(${patch.price}::numeric, price),
        status = COALESCE(${patch.status}, status),
        notes = COALESCE(${patch.notes}, notes)
      WHERE id = ${id}::uuid
      RETURNING *
    `;

    if (body.status === "done") {
      const [existing] = await db.sql`SELECT id FROM invoices WHERE job_id =
      if (!existing) await createInvoiceForJob(job);
    }
    return Response.json(job);
  }
}

```

```

    if (resource === "invoices") {
      const patch = {
        tax: body.tax ?? null,
        paymentMethod: body.paymentMethod ?? null,
        paid: body.paid ?? null,
      };
      const [invoice] = await db.sql`
        UPDATE invoices SET
          tax = COALESCE(${patch.tax}::numeric, tax),
          payment_method = COALESCE(${patch.paymentMethod}, payment_method),
          paid = COALESCE(${patch.paid}::boolean, paid)
        WHERE id = ${id}::uuid
        RETURNING *
      `;
      return Response.json(invoice);
    }

    return new Response("Unknown resource", { status: 400 });
  }

  if (req.method === "DELETE") {
    const id = url.searchParams.get("id");
    if (resource === "clients") {
      await db.sql`DELETE FROM clients WHERE id = ${id}::uuid`;
    } else if (resource === "jobs") {
      await db.sql`DELETE FROM jobs WHERE id = ${id}::uuid`;
    }
    return Response.json({ ok: true });
  }

  return new Response("Method not allowed", { status: 405 });
} catch (err) {
  console.error(err);
  return new Response(JSON.stringify({ error: String(err) }), { status: 500 });
}
};

async function createInvoiceForJob(job) {
  const [{ next_number }] = await db.sql`
    UPDATE invoice_sequence SET next_number = next_number + 1
    WHERE id = 1
    RETURNING next_number - 1 AS next_number
  `;
  const number = makeInvoiceNumber(next_number);
  await db.sql`
    INSERT INTO invoices (job_id, client_id, number, subtotal, tax, payment_metho

```

```
VALUES (${job.id}::uuid, ${job.client_id}::uuid, ${number}, ${job.price}::num
`;  
}  
  
export const config = { path: "/api/owner" };
```